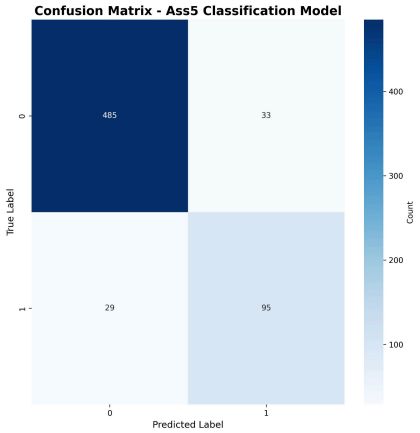
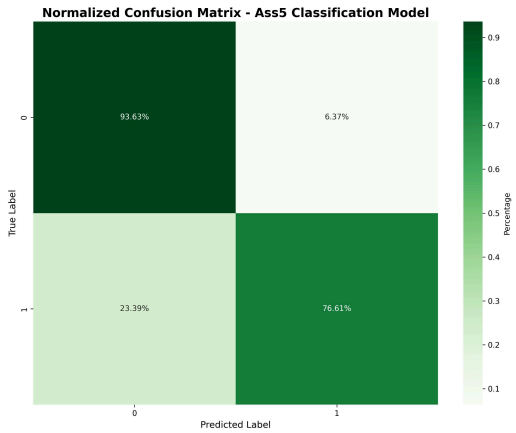
6758003756 Kijputhip Oonsonk





Results:

Data preparation (how you have prepared the data, how you split the data etc)

I started with ass5data.csv, which has 3,208 rows and 19 columns. The target is ProdTaken (whether the customer took the product: 0 or 1). I split this data into train.csv (2,566 rows) and test.csv (642 rows) using a stratified 80/20 split so both sets keep the same class ratio. Before training, I cleaned text values by trimming spaces and fixing label inconsistencies (for example, Fe Male to Female, Free Lancer to Freelancer, and Unmarried to Single). Inside training, train.csv was split again into training and validation sets (80/20), and the model used class weights to handle class imbalance.

Analysis (What you observe in the data)

The dataset is imbalanced: class 0 is about 80.7% and class 1 is about 19.3% in full, train, and test data, so the model must avoid just predicting class 0 all the time. The features are mixed: customer profile, travel behavior, and sales-contact details (for example age, occupation, number of follow-ups, and monthly income). In the current CSV files, there are no missing values, but the pipeline still includes imputers so it remains safe if missing values appear later. This is a binary classification problem where the main goal is to correctly identify customers likely to take the product.

Feature extraction (How you come to the conclusion on what kind of feature you want to create)

I used two feature paths based on data type. Numerical columns (12 columns) were median-imputed and standardized, while categorical columns (6 columns) were most-frequent-imputed and one-hot encoded. This choice was made because neural networks train better when numeric inputs are scaled, and one-hot encoding lets the model use category information without forcing fake numeric order. After preprocessing, the model receives 33 input features (12 numeric + 21 one-hot category levels).

Building model (How you build the model: hidden layers, optimizers, loss functions, etc)

The model is a feed-forward neural network built with TensorFlow/Keras. It uses hidden layers of 512, 256, 128, 64, and 32 neurons with ReLU activation, plus Batch Normalization, Dropout (0.3 then 0.2), and L2 regularization to reduce overfitting. The output layer has 1 neuron with sigmoid activation for binary prediction probability. Training uses the Adam optimizer with

learning rate 0.001, batch size 32, and up to 100 epochs, with callbacks for checkpointing best model, early stopping, and learning-rate reduction.

Loss Function / Training Objective

The training objective is binary cross-entropy loss, which penalizes wrong probability predictions for class 0/1. In simple terms, the model is trained to output high probability for true positives and low probability for true negatives. During inference, a threshold of 0.5 is used: probability >= 0.5 predicts class 1, otherwise class 0. Because the data is imbalanced, class weights are applied during training so mistakes on minority-class samples have stronger impact.

Evaluation results (Results in accuracy and recall)

Using the latest test output in ass5_evaluation_metrics.txt, the model achieved accuracy = **0.903427** and recall = 0.766129 (with precision 0.742188, F1 0.753968, AUC 0.909920). The count-based confusion matrix is [[485, 33], [29, 95]], meaning: 485 true negatives, 33 false positives, 29 false negatives, and 95 true positives. The normalized confusion matrix (percentage by each true class row) is approximately [[93.63%, 6.37%], [23.39%, 76.61%]]: for true class 0, 93.63% were correctly predicted as 0; for true class 1, 76.61% were correctly predicted as 1. In simple language, the model is strong overall and very good at class 0, but it still misses about 23% of actual class 1 cases, which is the main area to improve.